Inteligencia Artificial

ENRIQUE PELÁEZ Ph.D.

Escuela Superior Politécnica del Litoral – ESPOL

Facultad de Ingeniería en Electricidad y Computación – FIEC

Clase # 21 - PAO 2024 – II

Agenda

- Técnicas para resolver problemas de IA
  - Estrategias de control de búsqueda, conducida por datos (data driven) y por objetivos (goal driven)
- Búsqueda a ciegas
  - Generar y probar (Generate and Test)
  - Breath Search
  - Depth Search
- Introducción a la Búsqueda Heurística

Técnicas para resolver problemas en IA

- Este ejemplo expone 3 características importantes de las técnicas utilizadas en IA:
  - **Búsqueda**: proporciona un mecanismo para resolver un problema donde no existe un método directo.
  - **Uso de conocimiento**: proporciona un mecanismo para resolver problemas complejos utilizando las estructuras e información existentes de los objetos involucrados.
  - **Abstracción**: proporciona un mecanismo para concentrarse en aquellos aspectos que son importantes para la solución del problema, y separa aquellos que no son importantes y que podrían hacer que el proceso colapse.

Técnicas para resolver problemas en IA

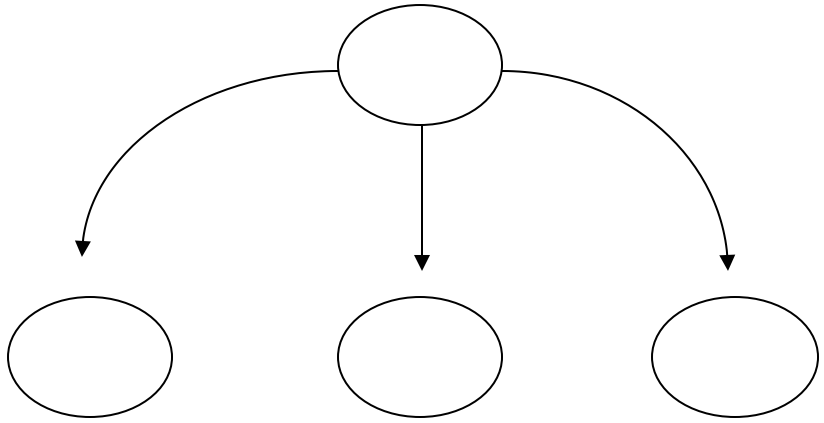
- Las primeras investigaciones en IA mostraron que el comportamiento inteligente necesita conocimiento.

- Y el conocimiento tiene algunas propiedades indeseables, como:

  - Es voluminoso
  - Difícil de caracterizar
  - Cambia
  - Comúnmente se organiza de la misma manera como se usará.

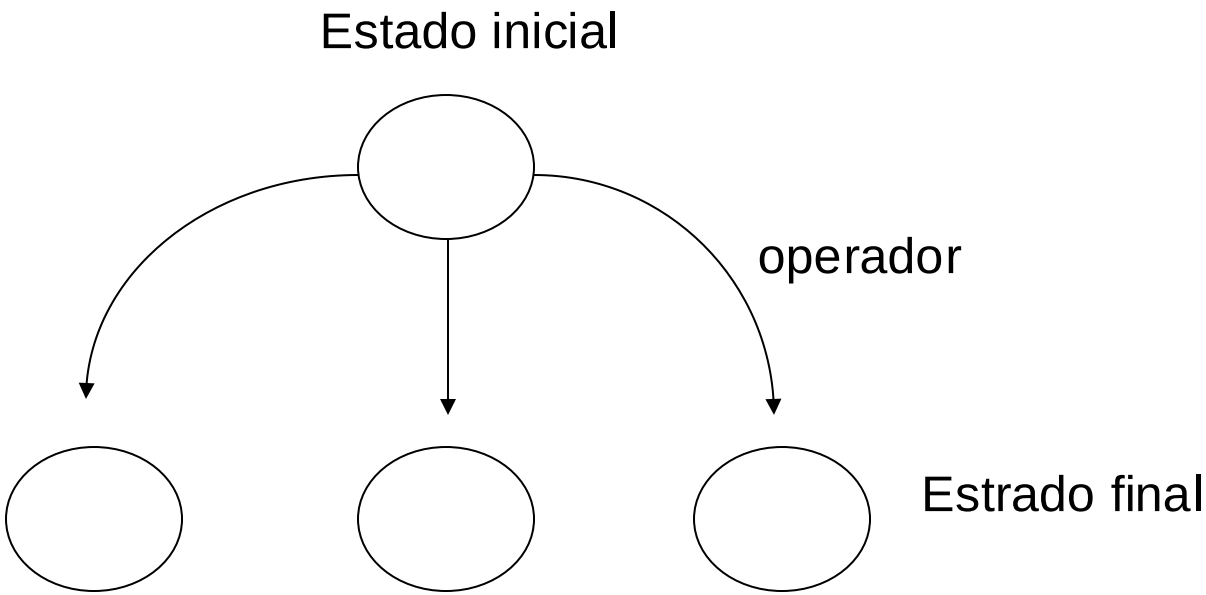
Técnicas para resolver problemas en IA

- Técnicas de IA en general, incorporan procesos de búsqueda.
- Los métodos de búsqueda pueden considerarse como árboles de búsqueda, en los que cada nodo representa un estado del problema y cada enlace representa la relación entre los estados vinculados.
- En general, hay reglas que determinan cómo navegar por el árbol, y partes del árbol pueden generarse o expandirse a medida que se activan las reglas.



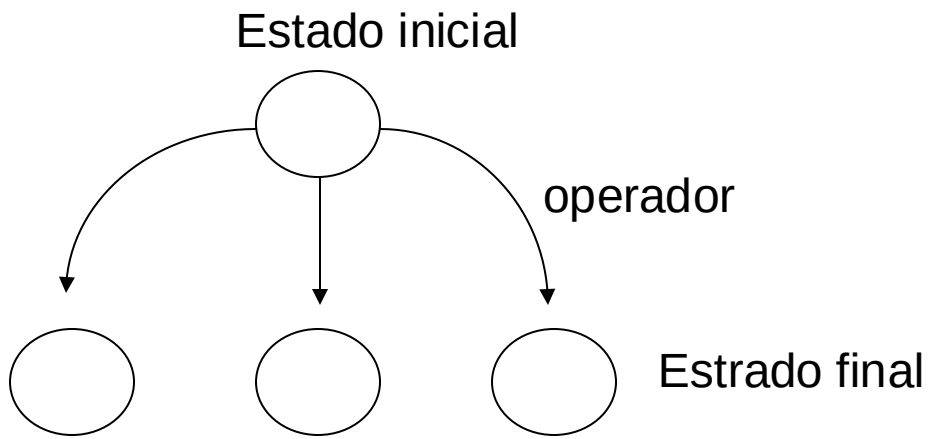
Técnicas para resolver problemas en IA

- Búsqueda de estados



Técnicas para resolver problemas en IA

- Las reglas describen las acciones u operadores que permiten pasar de un estado a otro.
- Una regla está compuesta por un lado izquierdo y derecho
- El lado izquierdo determina la aplicabilidad de la regla; es decir, describa los estados a los que se pueden aplicar las reglas.
- El lado derecho describe la operación / acción que tiene lugar si se aplica la regla (se activa); es decir, pasar al siguiente estado.



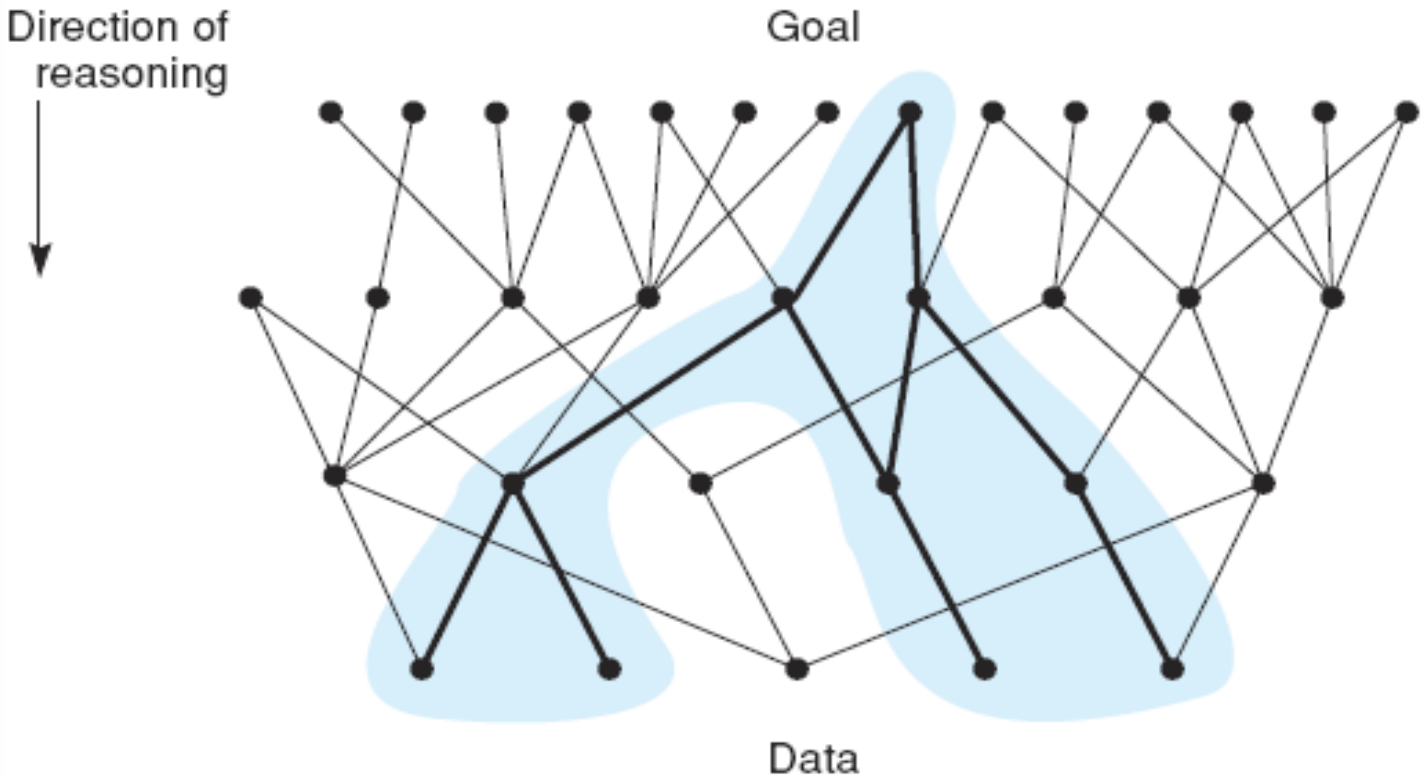
Técnicas para resolver problemas en IA

- Estrategias de búsqueda en el espacio de posibles estados
  - Goal-driven - encadenamiento hacia atrás
    - Toma el objetivo a ser resuelto. Ve qué reglas o movimientos legales podrían usarse para generar este objetivo y determina qué condiciones deben ser ciertas para usarlos.
    - Estas nuevas condiciones se convierten en los nuevos objetivos para la búsqueda, se remonta a los hechos del problema.
    - Esto encuentra la cadena de movimientos o reglas que conducen de los datos a la meta, pero al revés.

Por ejemplo: Similar a: IF ….. THEN ……

Técnicas para resolver problemas en IA

- Goal-driven

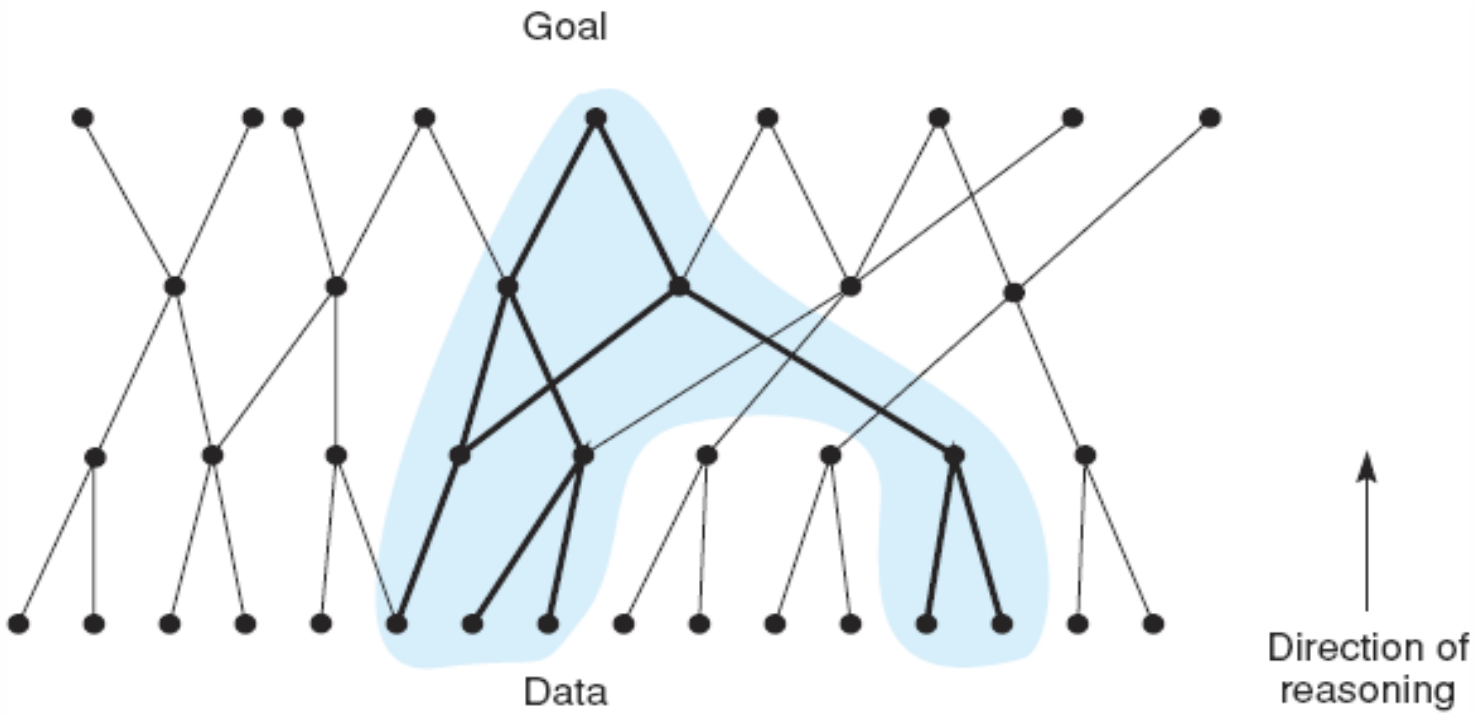


Técnicas para resolver problemas en IA

- Data-driven: encadenamiento hacia adelante
  - La búsqueda comienza con los hechos dados del problema y un conjunto de movimientos legales o reglas para cambiar el estado.
  - La búsqueda continúa aplicando reglas a los hechos para producir nuevos hechos, hasta que genera un camino que satisface la condición del objetivo.

Técnicas para resolver problemas en IA

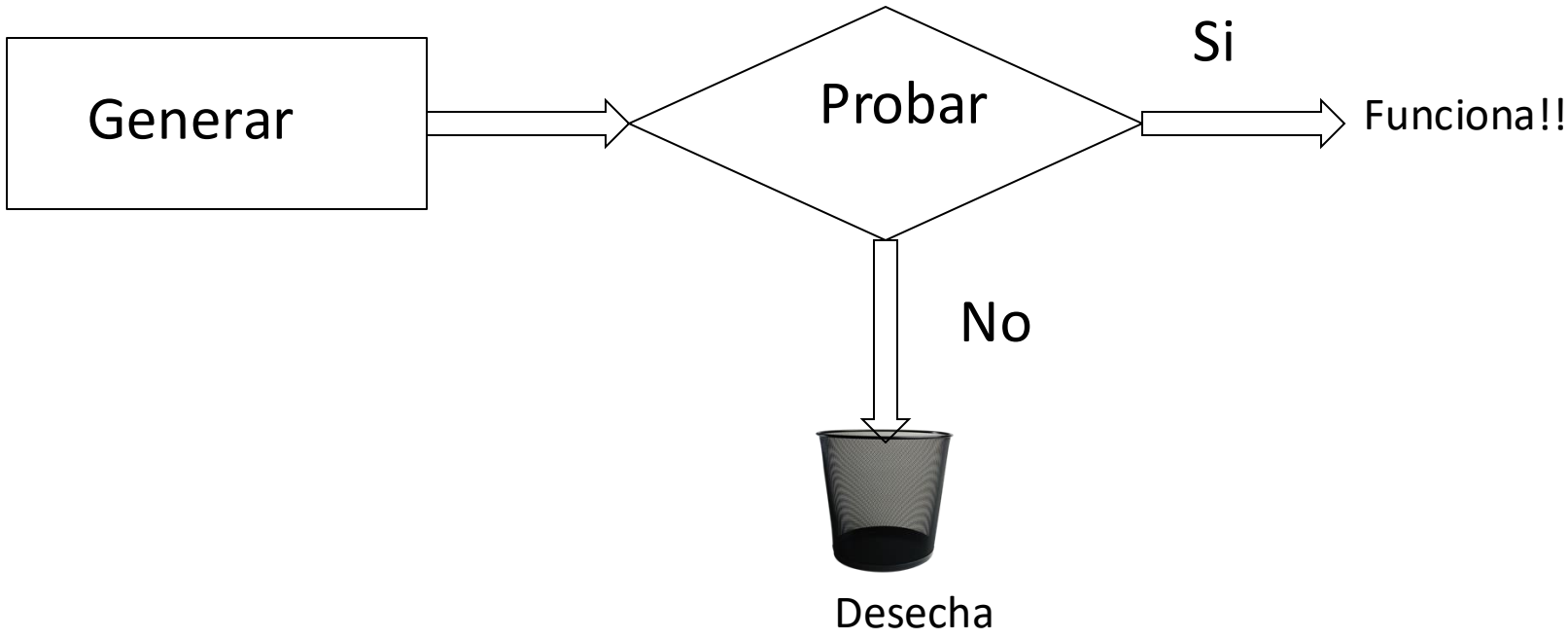
- Data Driven



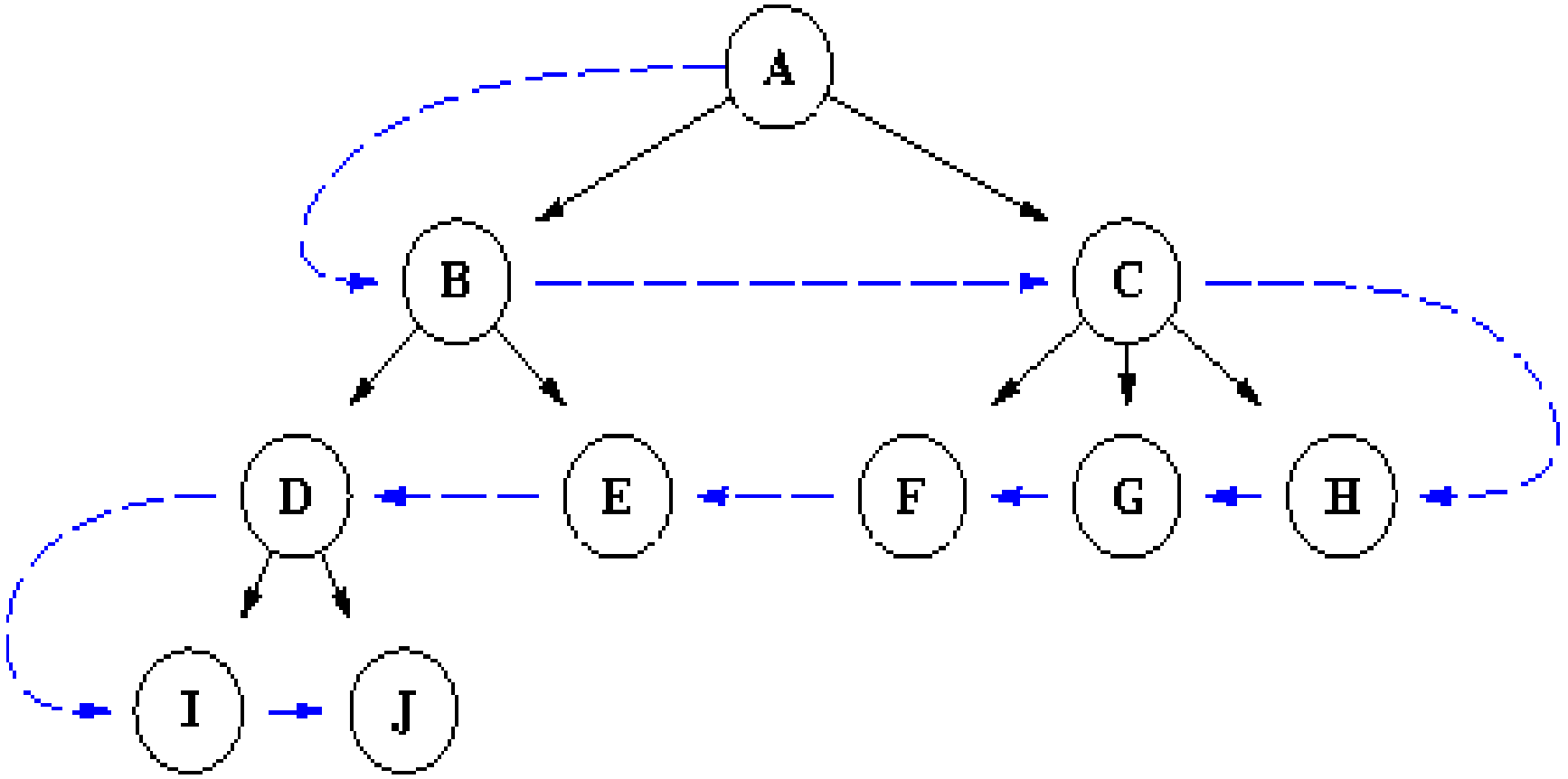
Técnicas para resolver problemas en IA

- Estrategias de búsqueda a ciegas:
    - Generate and Test
    - Breadth First
    - Depth First
    - Uniform cost search
    - Limited depth
    - Interactive depth
    - Bidirectional search

Generate and Test



Breath - First

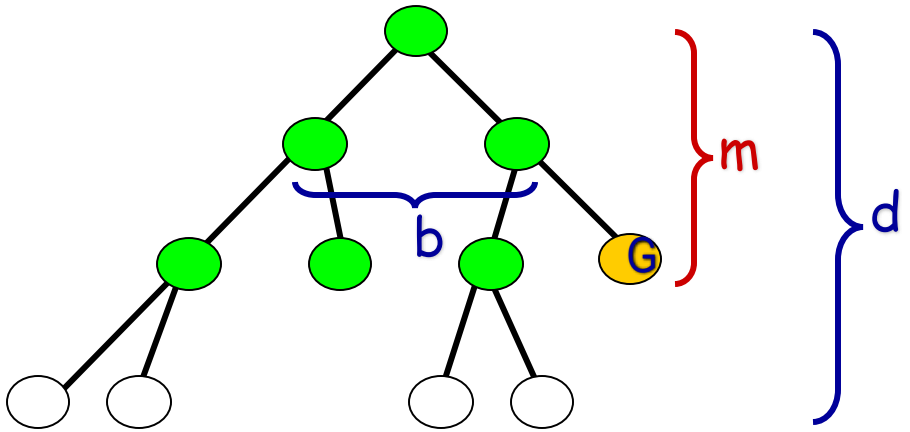


Breath - First

```
Begin
    open := [Start];
    closed := [ ];
    while open π [ ] do
        begin
            remove leftmost state from open, call it X;
                if X is a goal then returns SUCCESS
                    else begin
                        generate children of X;
                        put X on closed;
                        discard children of X if already on open or closed;
                        put remaining children on right end of open
                    end
        end
    return FAIL
end
```

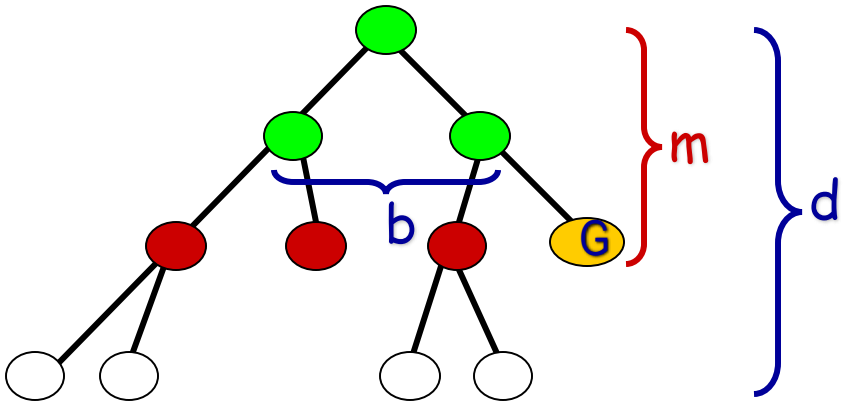
Breath - First

- Completo
  - ✓Si, (inclusive para arboles infinitos)
  - ✓Siempre encuentra la ruta más corta
- Tiempo
  - ✓$O(b^m)$

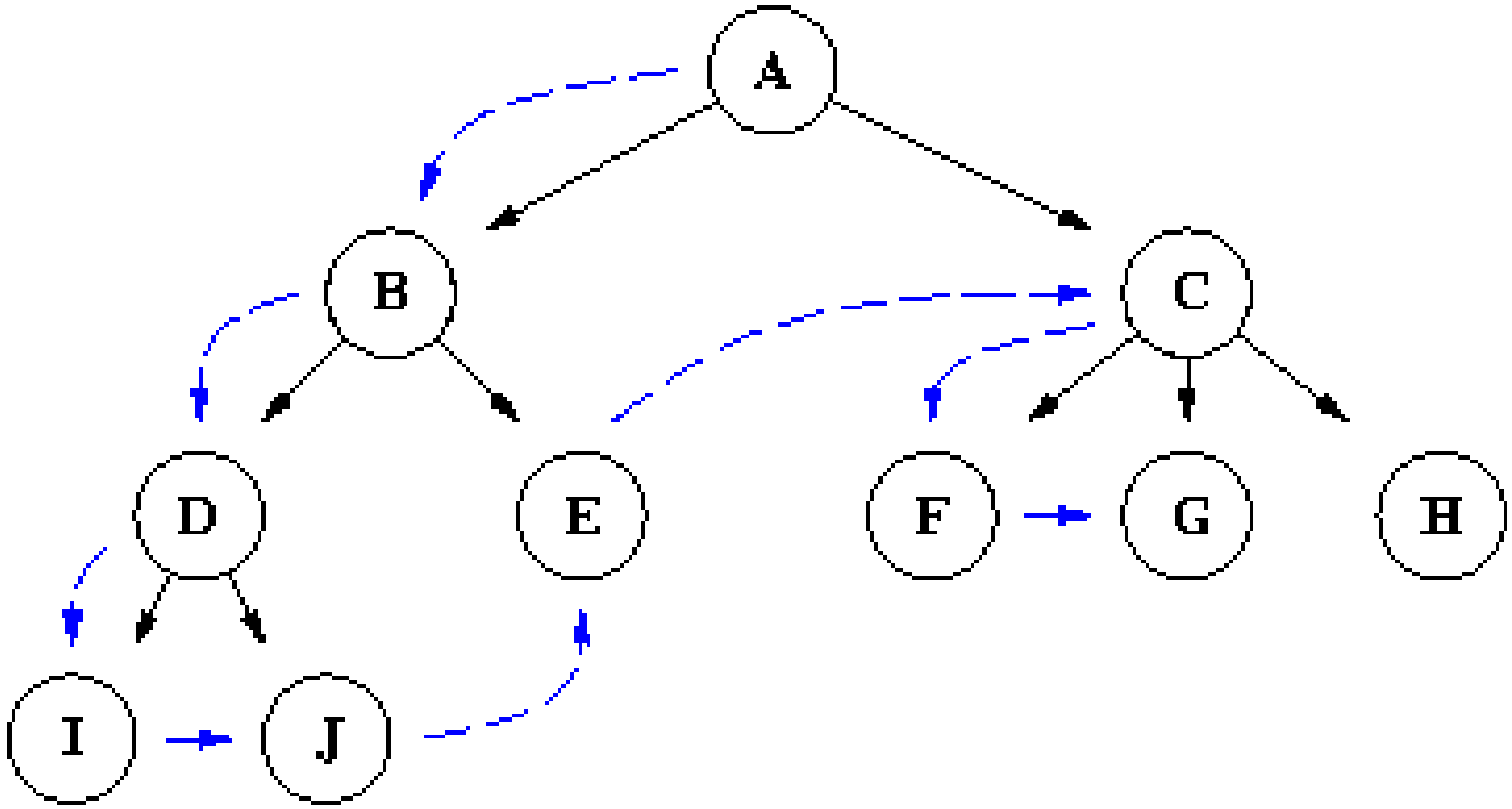


Breath - First

- Memoria
  - El número máximo de nodos se alcanza en el nivel "m" (objetivo)
  - En general: b^m
  - Generalmente "peor" que la profundidad primero



Depth - First

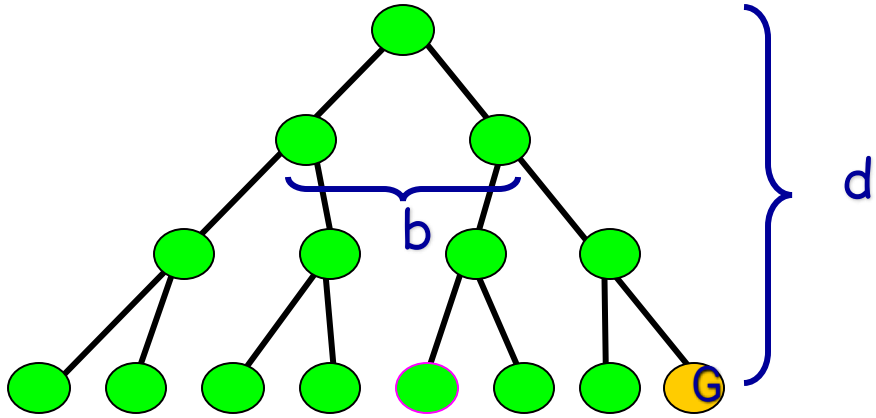


Depth - First

```
Begin
    open := [Start];
    closed := [ ];
    while open π [ ] do
        begin
            remove leftmost state from open, call it X;
                if X is a goal then returns SUCCESS
                    else begin
                        generate children of X;
                        put X on closed;
                        discard children of X if already on open or closed;
                        put remaining children on left end of open
                    end
        end
    return FAIL
end
```

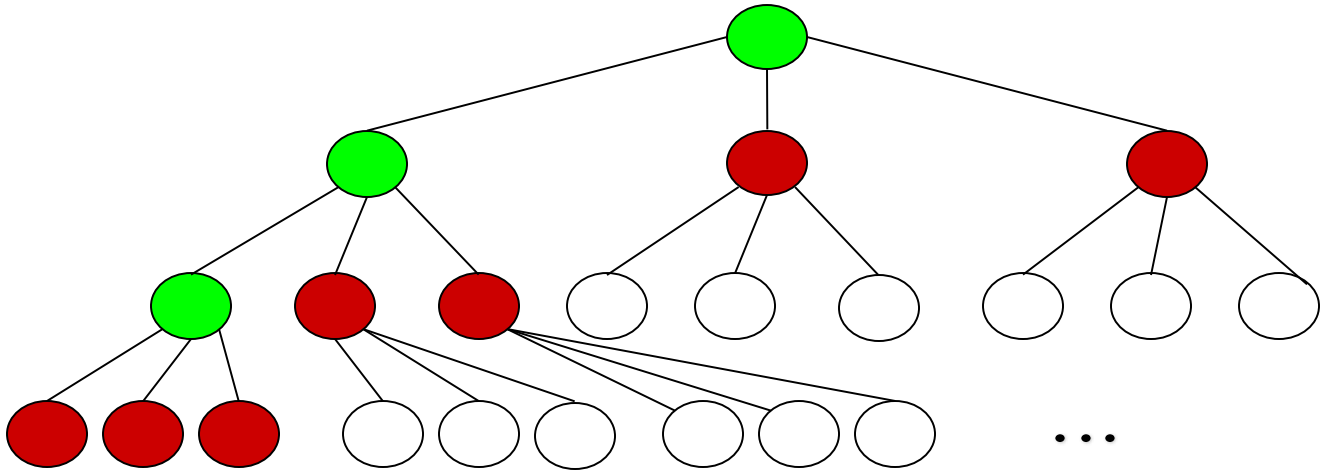
Depth - First

- Completo
  - Si -> árboles infinitos
- Tiempo
  - En el peor de los casos, el estado final podría ser la última rama del árbol.
  - $O(b^d)$



Depth - First

- Memoria
  - ✓En general: O(d*b)



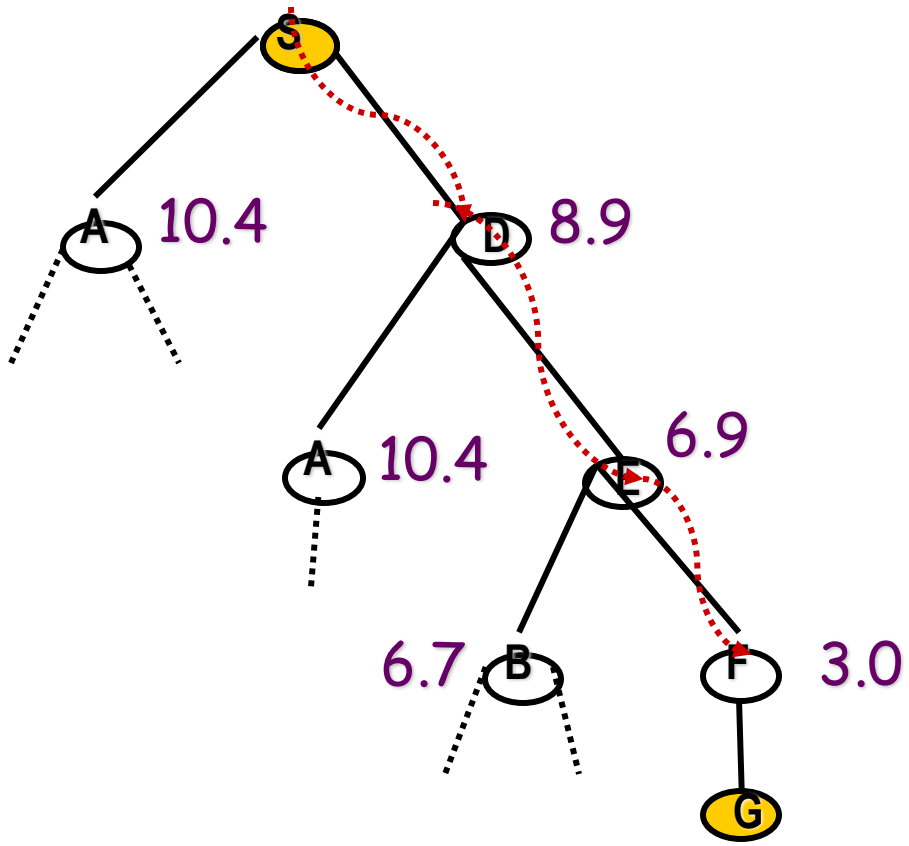
Búsqueda heurística

- Búsqueda heurística (o búsqueda informada)
  - Del griego "heuriskein", que significa descubrimiento o hallazgo.
  - La heurística ayuda o proporciona información para guiar el proceso de búsqueda.
  - La búsqueda basada en heurística, no siempre garantiza encontrar el resultado más óptimo (la mejor solución de todas), pero obtendremos muy buenos resultados en un tiempo razonable.
  - La heurística se usa para resolver problemas complejos donde generalmente tenemos una explosión combinatoria, donde los algoritmos de búsqueda exhaustivos tienen costos inaceptables.

Búsqueda heurística

- Entonces, utilizamos heurística para ”seleccionar" qué nodo expandir o explorar a continuación.
- La información heurística aparece como una función, que se evalúa con base en información específica para el contexto o problema en cuestión.
- Por lo tanto, la solución del problema de búsqueda consiste en maximizar o minimizar una función.
- La función heurística nos permite evaluar un nodo "localmente", en función del costo estimado para alcanzar el objetivo desde el nodo actual.
- Pero, hay algunos retos en la búsqueda con heurística :
  - La heurística podría no ayudar a encontrar la mejor solución: problema abierto. Sin embargo, el algoritmo A * lo resuelve con una función heurística h*
  - …

Búsqueda heurística



- Es como la búsqueda primero a lo profundo, pero:
- En lugar de seleccionar el siguiente nodo de izquierda o derecha,
- Selecciona el nodo con el mejor valor heurístico.